



DATA STRUCTURES – ICT1305

P.S. Palliyaguruge

1



ICT1403 – DATA STRUCTURES

- ✖ Syllabi
- ✖ **Data structures:** Random Access and Sequential access, Performance Analysis, Big O notation, Arrays, Array based Lists, **Linked Lists:** Array Based, Singly Linked, Doubly Linked, Circular, Sorted. Stacks using Arrays and Linked Lists, Queues using Arrays and Linked Lists, Hash Tables using Arrays and Linked Lists, Trees using Arrays and Linked Lists, Graphs: Depth First Search, Breadth First Search .
- ✖ **Simple Sorting & Searching Algorithms:** Selection sort, Bubble sort, Merge sort, Binary search.

2

ICT1403 – DATA STRUCTURES



× References

- + Algorithms, Data Structures, and Problems - Addison Wesley.
- + Data Structures, Algorithms, And Object Oriented Programming – Gregory L. Heileman
- + Waite Group's Object-Oriented Programming in C++, Robert Lafore
- + Fundamentals of Data Structures in C++

3

ICT1403 – DATA STRUCTURES



× Intended Learning Outcomes:

- × At the end of this course students should be able to
 - + demonstrate applications of different Data Structures.
 - + identify advantages and disadvantages of different Data Structures.
 - + demonstrate the knowledge of various common algorithms.
 - + select appropriate Data Structures in developing computer applications.

4

ICT1403 – DATA STRUCTURES



❖ Prerequisites:

- + Principle of program design and programming – ICT1402
 - × Data types
 - × Structures in C
 - × Arrays
 - × Pointers
 - × Functions
 - × File handling

5

INTRODUCTION



Software engineering: The study of ways in which to create large and complex computer applications (involve many programmers and designers) based on the needs and requirements of end users.

- Involves the full life cycle of a software project (SDLC)
 - Specification, requirements gathering, design, verification, coding, testing, quality assurance, user acceptance testing, production, and ongoing maintenance.

6

INTRODUCTION



Having an in-depth understanding on every component of software engineering is not mandatory, however, *it is important to understand that the subject of **data structures and algorithms** is concerned with the coding phase.* The use of data structures and algorithms is the nuts-and-blots used by programmers to store and manipulate data.

7

WHAT ARE DATA STRUCTURES?



- Data Structures are ways in which data are arranged in Computer's memory or stored in hard disk.
- In computer science, a **data structure** is a way of storing data in computer so that it can be used efficiently.
- A carefully chosen data structure will allow a more efficient algorithm to be used



8

WHAT ARE DATA STRUCTURES?



- A well-designed data structure allows a variety of critical operations to be performed, using as few resources (execution time and memory space) as possible
- Data Structures include Arrays, linked lists, stacks, queues, trees, graphs, tables, files, etc..
- The Data Structures can be implemented using **abstract data types**



9

WHAT ARE DATA STRUCTURES?



Abstract data type

In computing, an **abstract data type (ADT)** is a specification of a set of data and the set of operations that can be performed on the data.

10

WHAT ARE DATA STRUCTURES?



Example of Data Abstraction

- One example of data abstraction is the use of the data type ***double***, which is an abstraction of real numbers.
- The computer hardware limits the range of real numbers that can be represented.
- Different computers use variety of representation schemes for type double.
- We can use the data type ***double*** and its associated operators (+, -, *, /, =, ==, <, and so on) without being concerned with the details of its implementation.

11

WHAT ARE DATA STRUCTURES?



Abstract:

- Independent of various concrete implementations.
- The definition can be mathematical, or it can be programmed as an interface.
 - Interface provides a *constructor*, which returns an abstract handle to new data, and several *operations*, which are functions accepting the abstract handle as an argument.

12

WHAT ARE DATA STRUCTURES?



Abstract data type

A way of looking at a data structure: focusing on what it does and ignoring how it does its job.

- A stack or a queue is an example of an ADT.
 - Both stacks and queues can be implemented using an array. It is also possible to implement stacks and queues using a linked list. This demonstrates the **"abstract" nature** of stacks and queues: how they can be considered separately from their implementation.

13

WHAT ARE ALGORITHMS?



- Algorithms are procedures; a software program uses to manipulate the data in these data structures
- An **algorithm** is a procedure (a finite set of well-defined instructions) for accomplishing some task which, given an initial state, will terminate in a defined end-state.
- The computational complexity and efficient implementation of the algorithm are important in computing, and this depends on suitable data structures.

14

WHAT ARE ALGORITHMS?



Algorithms are used to manipulate the data contained in these data structures as in searching and sorting.

Many algorithms apply directly to a specific data structures.

- How to insert new data, search for a specified item, and deleting a specific item.

15

WHAT ARE ALGORITHMS?



Commonly used algorithms include are useful for:

- Searching for a particular data item (or record).
- Sorting the data. There are many ways to sort data.
Simple sorting, Advanced sorting
- Iterating through all the items in a data structure.
(Visiting each item in turn so as to display it or perform some other action on these items).

16

WHAT FOR DATA STRUCTURES AND ALGORITHMS?



- Real world data storage
 - Data that describes physical entities external to the computer
 - E.g.: A Personnel record, Stack of index cards

17

WHAT FOR DATA STRUCTURES AND ALGORITHMS?



- Programmers tools
 - Not all DSs are used to store real-world data.
 - Some DSs are not meant to be accessed by the user, but by the program itself
- Real world modeling
 - Some DSs directly model a real-world situation
 - Eg: Graphs to represent air line routes, tasks in a project, Queues to represent a queue at a clinic, etc..

18

AN OVER VIEW OF ALGORITHMS



- Many of algorithms apply directly to specific data structures. For most DSs you need to know how to
 - Inset new data item
 - Search for a specified item
 - Delete a specified item
 - Iterate/Traverse each element

Efficiency

In selecting an appropriate data structure for a particular application it is important to know and compare the efficiency of the Data Structures

19

ASYMPTOTIC NOTATION - BIG O NOTATION



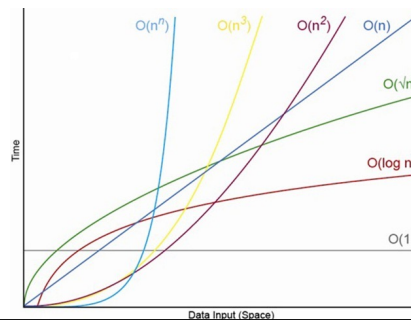
- A problem may have numerous algorithmic solutions. In order to choose the best algorithm for a particular task, you need to be able to judge how long a particular solution will take to run.
- Need some way to compare algorithms against one another.
- **Asymptotic complexity** is a way of expressing the *main component* of the cost of an algorithm, using idealized units of computational work.

20

ASYMPTOTIC NOTATION - BIG O NOTATION



Is a mathematical notation used to describe the asymptotic behavior of functions. Its purpose is to characterize a function's behavior for very large (or very small) inputs in a simple but rigorous way that enables comparison to other functions



21

ASYMPTOTIC NOTATION - BIG O NOTATION



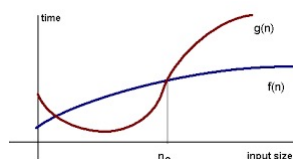
Eg:

Some might Say that “Algorithm ‘A’ is twice as fast as algorithm ‘B’”

What happens if you increase the number of items by 50% - “Algorithm ‘A’ is three time as fast as algorithm ‘B’”

If you have half as many items - “Algorithm ‘A’ and ‘B’ are now equal”

i.e. Need a comparison that’s related to the number of items



22

BIG O NOTATION



Insertion in an unordered Array:

- Time taken to insert an item to an array is constant
- The time does not depend on the number of items
- Always time $T=k$

Linear Search

- Proportional to number of items $N/2$
- $T \propto N/2 \rightarrow T=K*N/2 \rightarrow T=KN$

Binary Search

- Proportional to $\log(N)$
- $T \propto \log_2(N) \rightarrow T=K*\log_2(N) \rightarrow T=K*\log(N)$

23

BIG O NOTATION



Multiply a ***n*-digits** number with a ***m*-digit** number:

- Two method
 - Repeated Additions
Requires maximum $\sim 10^n(n+m)$ additions
(since max *n*-digit number $\sim 10^n$)
 - Digit by digit multiplication (with help of a multiplication table)
Requires maximum $\sim 2m(n+m)$

Which is faster? When $n=m$

	$g(n)$	$f(n)$
$n = m$	$10^n(n+m)$	$2m(n+m)$
1	20	4
2	400	16
3	6,000	36
4	80,000	64

24

C/C++ BACKGORUND



Input/Output

printf/scanf – stdio.h : standard input/output

Contional Expressions: if, if..else

Control Structures : for, while

File input/output –stdito.h

Functions

Parameter passing

Pointers to functions

Recursive function

Pointers: A Special type of variable that stores memory addresses of other variables. Useful when creating dynamic memory allocation.

25

C/C++ BACKGORUND



Pointers: A Special type of variable that stores memory addresses of other variables. Useful when creating dynamic memory allocation.

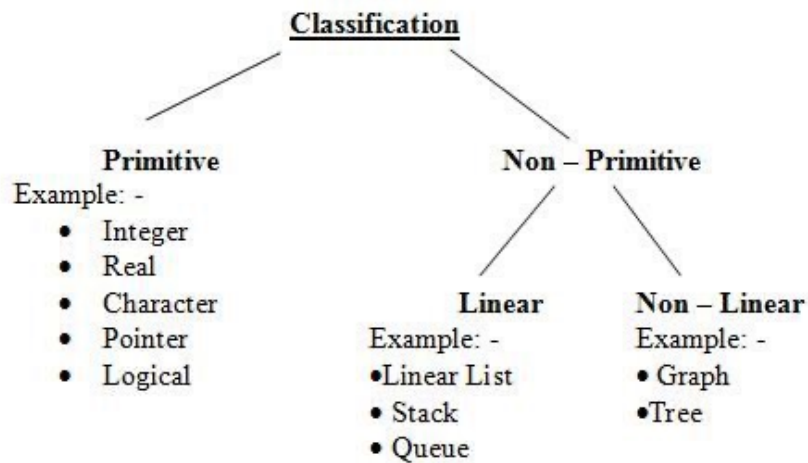
Eg:

```
int *num,*data;
...
...
data=malloc(sizeof(int)*10);
...
num=data;
...
free(data);
```

```
int *num,*data;
...
...
data=new int [10];//c++
...
num=data;
...
delete [] data;//c++
```

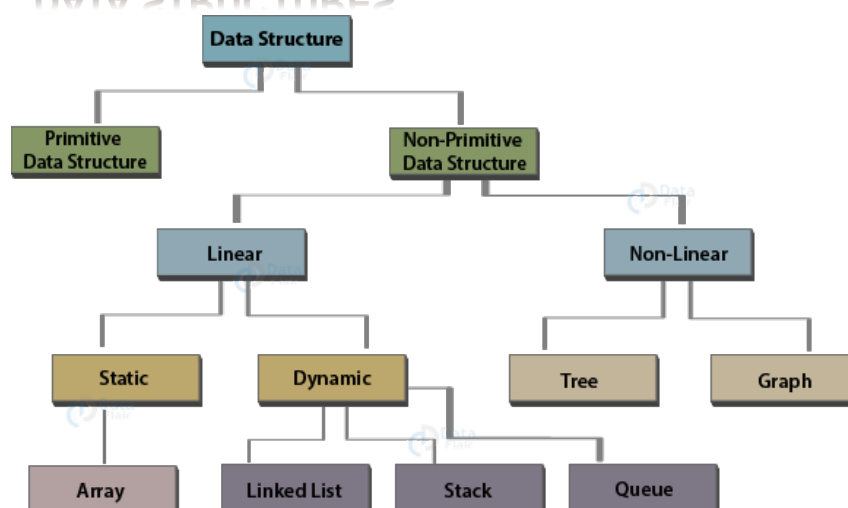
26

DATA STRUCTURES



27

DATA STRUCTURES



28

ARRAYS



Definition: Arrays are derived data types that store multiple values under the same name

- In computer programming, a group of homogeneous elements of a specific data type is known as an array, one of the simplest data structures
- C/C++ Arrays are the data structures which can be used to store consecutive values of the same **data types**
- All the values are stored in consecutive memory locations
- The values can be accessed by using the position of the stored value

29

ARRAYS



- The Computer stores arrays in consecutive memory locations
- This makes the arrays' accessibility faster
- Unlike other data structures like Linked list, queue etc., the access to the elements is very fast
- Arrays can be multi-dimensional. Most cases a maximum of 3 dimensions will suffice for programming

30

ARRAYS



Declaring C/C++ Arrays:

The c/c++ arrays are declared with the data type name and the number of elements inside the square brackets.

Eg: `int var_name[50];` // C++ Array of type int with maximum size = 50.

The above declaration means, the var_name is an array of integer type. The values inside this var_name array can be accessed by referring to the position of their elements like var_name[0], var_name[1] etc.,

31

ARRAYS



All the C/C++ arrays are based on Zero index values. The position reference starts at 0 and counts till the n-1th position.

```
var_name[0] = 0;
```

```
var_name[1] = 1;
```

```
var_name[2] = 30;
```

```
cout << "var_name[0] =" << var_name[0] ; //C++
```

The position reference should not exceed the maximum size specified in the declaration.

Multi dimensional arrays?

32

ARRAYS



Advantages

- Quick insertion
- Very fast access if index is known

Disadvantages

- Slow search
- slow deletion
- fixed size

33

POINTERS AND ARRAYS



Pointers

- Special data types that store the address of values as oppose to the value itself
- A Special type of variable that stores memory addresses of other variables. Useful when creating dynamic memory allocation.
- A pointer actually points to the **block of memory** where a specific value is stored
- Allows programmer to describe where data is stored

Declaration (C/C++)

```
dataType    *variableName; Or
dataType*   variableName;
```

34

POINTERS AND ARRAYS



Pointer variable declaration in C/C++

```
dataType *variableName; Or
dataType* variableName;
```

Eg: float *ptrX;

here

**ptrX* is of type float

ptrX represents a memory address

What about *ptrX+1 and ptrX+1? (Pointer arithmetic)

35

POINTERS AND ARRAYS



Relationship

The name of an array is synonymous with a pointer to the memory location of the first element in the array

Eg:

```
int *num,*data; // C
...
...
data=new int [10]; //C++
...
num=data;
...
delete [ ] data; //C++
```

36

LIST



- A flexible structure that can grow or shrink on demand
- Elements can be accessed, inserted or deleted at any position
- Lists can be concatenated or spilt
- Usage in applications
 - Information retrieval, programming language translation, simulation
- Definition
 - A list is a sequence of zero or more elements of a given type
 $a_1, a_2, a_3, \dots, a_n ; n \geq 0$,
 a_1 – first element, a_n – last element

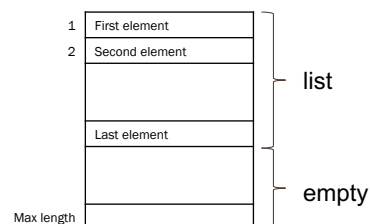
37

LIST



Array Implementation

- Elements are stored in contiguous cells of an array
- Easy to traverse
- New elements can be appended to tail
- Inserting elements into middle requires shifting elements one place over to make room
- Deleting any element except last also requires shifting elements to close up the gap

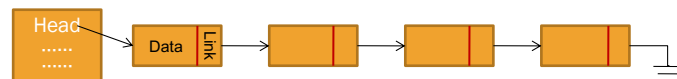


38

LINKED LIST



- One of the most fundamental data structures
- Store a collection of data items
- May be defined as **a dynamic ordered n-tuple**
- It consists of a sequence of nodes, each containing arbitrary data fields and one or two references ("links") pointing to the next and/or previous nodes

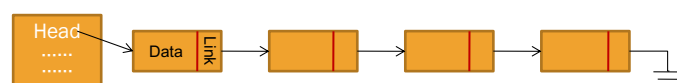


39

LINKED LIST



- **A self-referential data type - Contains a pointer or link to another data element of the same type**
- Uses pointer to link successive elements
- Allow to add or delete nodes at any point
- Sequential access
- Many other structures based on linked list



40

LINKED LIST



• Node

- A node consist with the data stored in the node and a pointer to the next node in the list

• Link's structure (C)

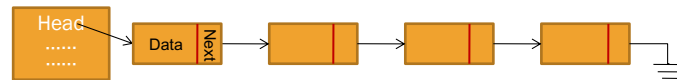
```
struct listNode
```

```
{
```

```
    int data;
```

```
    struct listNode *link; //address of the  
                           next node
```

```
};
```



41

LINKED LIST



• LinkedList Structure

• Operations?

```
struct linkedList{
```

```
    struct listNode * head; /* The first element of the list */
```

```
    int length; /* Keeps the number of items */
```

```
}
```

Common operations

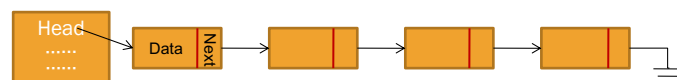
void initList(...) – to initialize the list

void insert(...); - A method to add an item

void display(...); - to display the contents of the list

void delete(...); - to remove a given item from the list

Link* find(...); - to search for a given key item



42

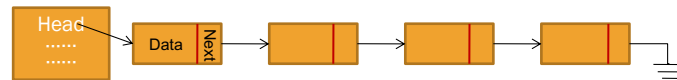
LINKED LIST



•Implementing A Linked list

- Create an empty list
- head is NULL and length=0

```
struct linkedList initList()
{
    struct linkedList L=malloc(sizeof(struct linkedList ));
    L->head=NULL;
    L->length=NULL;
    return L;
}
```



43

LINKED LIST



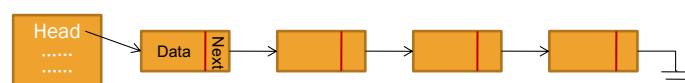
•Implementing A Linked list

- Inserting an item

This insertion method adds a new node at the end of the existing list.
Time taken to insert a node is proportional to the number of elements in existing list. i.e. $O(n)$

To do: Suggest a better way to insert a new node at the end

- Create a new Node
- If the Linked List is empty => add the new node as the Head of the Linked List
- Else find the last node and make next of the last node to the new node



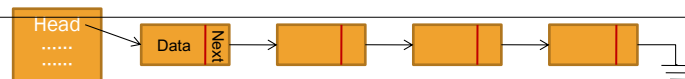
44

LINKED LIST



•Implementing A Linked list •Inserting an item

```
void insert(int newData, struct linkedList *L)
{
    struct listNode *newNode=malloc(sizeof(struct listNode)); //Create an entry node
    newNode->data=newData;
    newNode->next=NULL;
    if (L->head==NULL)
        L->head=newNode;
    else
    {
        struct listNode *scan=head;
        while (scan->next != NULL)
            scan=scan->next;
        scan->next=newNode;
    }
    L->length++;
}
```



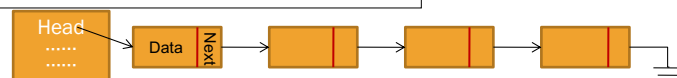
45

LINKED LIST



•Implementing A Linked list •Delete an item

```
void deleteItem (int Ddata, struct linkedList *L)
{
    struct listNode *scan1=L->head;
    if (scan1->data== Ddata) {
        L->head=L->head->next;
        free(scan1);
        L->length--;
    }
    else {
        Link *scan2=scan1;
        while (scan1->data!=Ddata && scan2->next
!=NULL) {
            scan1=scan2
            scan2=scan2->next;
        }
        if (scan1->data==Ddata){
            scan1->next=scan2->next;
            free( scan2);
            L->length--;
        }
        else
            printf("\n Item not found");
    }
}
```



46

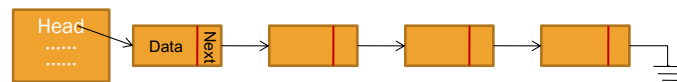
LINKED LIST



•Implementing A Linked list

•Display

```
void display ( struct linkedList L)
{
    listNode *current=L.head;
    while (current!=null) {
        printf("%d ->", current->data);
        current=current->next;
    }
    printf("End .\n");
}
```



47

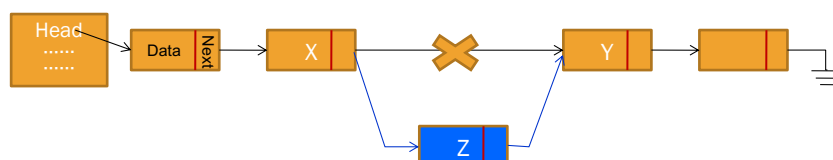
LINKED LIST



•Sorted Linked list

- A Linked list which data elements are inserted in ascending order of their key values.
- If i and j are two nodes s.t. $\text{Position}(j) > \text{Position}(i)$ then $\text{key}(j) > \text{key}(i)$

Insert into a sorted linked list



48

LINKED LIST



Linked lists vs. arrays

	Array	Linked list
Indexing	$O(1)$	$O(n)$
Inserting / Deleting at end	$O(1)$	$O(1)$ or $O(n)$
Inserting / Deleting in middle (with iterator)	$O(n)$	$O(1)$
Persistent	No	Singly yes
Locality	Great	Bad

49

LINKED LIST



Linked lists vs. arrays

- Elements can be inserted into linked lists indefinitely,
- An array will eventually either fill up or need to be resized
- Arrays allow random access, while linked lists allow only sequential access to elements.
- Singly-linked lists, in fact, can only be traversed in one direction.
- Linked lists require extra storage for references

A number of linked list variants exist that aim to restructure some of the above problems.

50

LINKED LIST



Linearly-linked list, Singly-linked list

The simplest kind of linked list is a **singly-linked list** (or **slist**), which has one link per node. This link points to the next node in the list, or to a **null** value

Doubly-linked list

A more sophisticated kind of linked list is a **doubly-linked list** or **two-way linked list**. Each node has two links: one points to the previous node, or points to a **null** value or empty list if it is the first node; and one points to the next, or points to a **null** value or empty list if it is the final node.

More pointers to update, but also less information is needed, since we can use backwards pointers to observe preceding elements in the list

51

LINKED LIST



Doubly-linked vs. singly-linked

- Double-linked lists require more space per node
- Elementary operations are more expensive
- Easier to manipulate because they allow sequential access to the list in both directions.(can insert or delete a node in a constant number of operations given only that node's address. (Compared with singly-linked lists, which require the previous node's address in order to correctly insert or delete.)
- Cannot be used as persistent data structures.

52

LINKED LIST



Circularly-linked list

In a **circularly-linked list**, the first and final nodes are linked together. This can be done for both singly and doubly linked lists.

Begin at any node and follow the list in either direction until return to the original node
No beginning or end.

Singly-circularly-linked list

Doubly-circularly-linked list

53

LINKED LIST



Circularly-linked vs. linearly-linked

- Circular linked lists are most useful for describing naturally circular structures,
- Able to traverse the list starting at any point.
- Quick access to the first and last records through a single pointer
- Complexity of iteration

54